

#AI Assisted Coding

#Assignment –20.3

#Ch. Nithin Reddy || 2303A51550 || Batch:- 8

Task 1 – Password Strength & Validation Check

```
# Write a Python user registration program with strong password
validation.

# Requirements:
# - Minimum 8 characters
# - At least one uppercase, lowercase, digit, and special character
# - Use regex for validation
# - Reject common passwords (e.g., 123456, password)
# - Keep asking until a valid password is entered
# - Print "Registration Successful"

import re

def is_strong_password(password):
    """
    Checks if a password meets the strong password requirements.
    """
    # Requirement 1: Minimum 8 characters
    if len(password) < 8:
        print("Password must be at least 8 characters long.")
        return False

    # Requirement 2: At least one uppercase, lowercase, digit, and
    special character
    if not re.search(r"[A-Z]", password):
        print("Password must contain at least one uppercase letter.")
        return False
    if not re.search(r"[a-z]", password):
        print("Password must contain at least one lowercase letter.")
        return False
    if not re.search(r"[0-9]", password):
        print("Password must contain at least one digit.")
        return False
    # Regex for common special characters: !@#$%^&*()_+={}
    [!@#$%^&*()_+={} \[\] ; ' < > , . ? / \ | ~ -
    if not re.search(r"[!@#$%^&*()_+={} \[\] ; ' < > , . ? / \ | ~ -]",
password):
        print("Password must contain at least one special character.")
        return False

    # Requirement 3: Reject common passwords
    common_passwords = [
```

```

        "password", "123456", "qwerty", "12345678", "123456789",
        "12345", "p@ssword", "admin", "guest", "welcome"
    ]
    if password.lower() in common_passwords:
        print("Password is too common. Please choose a stronger one.")
        return False

    return True

while True:
    user_password = input("Enter your new password: ")
    if is_strong_password(user_password):
        print("Registration Successful!")
        break
    else:
        print("Please try again.")

```

```

Enter your new password: 123456
Password must be at least 8 characters long.
Please try again.
Enter your new password: 12345678
Password must contain at least one uppercase letter.
Please try again.
Enter your new password: Aiac@12345
Registration Successful!

```

Task 2 – Insecure File Upload Handling

```

# Create a secure Python Flask file upload application.

# Requirements:
# - Allow file upload via form
# - Validate file extensions (only allow .jpg, .png, .pdf)
# - Limit file size (e.g., max 2MB)
# - Use secure filename handling
# - Store files in a safe directory
# - Reject invalid files with proper messages
# - Follow security best practices

```

Secure File Upload Application using Flask

This application will provide a web interface to upload files with the following security considerations:

- **Allowed Extensions:** Only `.jpg`, `.png`, and `.pdf` files will be accepted.
- **File Size Limit:** Files larger than 2MB will be rejected.
- **Secure Filename Handling:** Filenames will be sanitized to prevent directory traversal attacks.
- **Safe Storage:** Uploaded files will be stored in a designated `uploads` directory.

```

import os
from flask import Flask, request, redirect, url_for, render_template,
flash
from werkzeug.utils import secure_filename

# Configuration
UPLOAD_FOLDER = 'uploads'
ALLOWED_EXTENSIONS = {'jpg', 'png', 'pdf'}
MAX_FILE_SIZE = 2 * 1024 * 1024 # 2 MB

app = Flask(__name__)
app.config['UPLOAD_FOLDER'] = UPLOAD_FOLDER
app.config['MAX_CONTENT_LENGTH'] = MAX_FILE_SIZE
app.secret_key = 'supersecretkey' # Replace with a strong, random key
in production

# Create the upload folder if it doesn't exist
if not os.path.exists(UPLOAD_FOLDER):
    os.makedirs(UPLOAD_FOLDER)

def allowed_file(filename):
    return '.' in filename and \
        filename.rsplit('.', 1)[1].lower() in ALLOWED_EXTENSIONS

@app.route('/', methods=['GET', 'POST'])
def upload_file():
    if request.method == 'POST':
        # Check if the post request has the file part
        if 'file' not in request.files:
            flash('No file part')
            return redirect(request.url)
        file = request.files['file']
        # If the user does not select a file, the browser submits an
        # empty file without a filename.
        if file.filename == '':
            flash('No selected file')
            return redirect(request.url)
        if file and allowed_file(file.filename):
            filename = secure_filename(file.filename)
            file.save(os.path.join(app.config['UPLOAD_FOLDER'],
filename))
            flash(f'File {filename} successfully uploaded')
            return redirect(url_for('upload_file'))
        else:
            flash('Invalid file type or file too large.')
            return redirect(request.url)
    return render_template('upload.html')

# Error handler for file size exceed
@app.errorhandler(413)

```

```
def request_entity_too_large(error):
    flash('File too large. Maximum size is 2MB.')
    return redirect(url_for('upload_file'))

if __name__ == '__main__':
    # In a Colab environment, running Flask requires special handling.
    # We'll use a simple HTTP server to serve the Flask app.
    # For local development, you'd typically use app.run(debug=True).
    print(f"Upload folder: {os.path.abspath(UPLOAD_FOLDER)}")
    print("To run the Flask app, you'll need to create the
'templates/upload.html' file.")
    print("Then, you can run this cell and access the app via a public
URL.")
    # Note: Running Flask directly in Colab often requires tools like
'ngrok' or 'flask-ngrok'
    # For this example, we'll assume you'll run it in a way that
provides external access.
    # For a simple test, you might copy the generated HTML into a
string and serve it directly,
    # but for a full application, the template method is standard.
```

Upload folder: /content/uploads
To run the Flask app, you'll need to create the
'templates/upload.html' file.
Then, you can run this cell and access the app via a public URL.

To make the Flask application work, you need to create an HTML template for the upload form. This template should be saved in a subdirectory named `templates` relative to where your Flask app is running.

```
# Create the 'templates' directory if it doesn't exist
TEMPLATES_DIR = 'templates'
if not os.path.exists(TEMPLATES_DIR):
    os.makedirs(TEMPLATES_DIR)

# Define the content of the HTML template
upload_html_content = '''
<!doctype html>
<title>Upload new File</title>
<h1>Upload new File</h1>
{% with messages = get_flashed_messages() %}
    {% if messages %}
        <ul class=flashes>
            {% for message in messages %}
                <li>{{ message }}</li>
            {% endfor %}
        </ul>
    {% endif %}
{% endwith %}
```

```

<form method=post enctype=multipart/form-data>
  <input type=file name=file>
  <input type=submit value=Upload>
</form>
'''

# Write the content to the upload.html file inside the templates
directory
with open(os.path.join(TEMPLATES_DIR, 'upload.html'), 'w') as f:
    f.write(upload_html_content)

print(f"Created '{TEMPLATES_DIR}/upload.html' successfully.")
print("You can now run the Flask application cell.")

Created 'templates/upload.html' successfully.
You can now run the Flask application cell.

```

How to run the Flask application in Colab:

1. **Run the previous two code cells.** This will set up the Flask application and create the `templates/upload.html` file.
2. **Install flask-ngrok** (or use ngrok directly) to expose your local Flask server to the internet, allowing you to access it from your browser. `python !pip install flask-ngrok`
3. **Modify the Flask app execution** to use flask-ngrok. You'll need to add from `flask_ngrok import run_with_ngrok` and call `run_with_ngrok(app)` before `app.run()`.

```

# Example of modified Flask app execution (add this to the main
app cell or a new cell)
from flask_ngrok import run_with_ngrok

# ... (your existing Flask app definition)

if __name__ == '__main__':
    run_with_ngrok(app) # Starts ngrok when app is run
    app.run()

```

After running this modified cell, flask-ngrok will provide a public URL that you can open in your browser to access the file upload form.

Task 3 – Real-Time Application: API Security Review

```

# Create a secure Python Flask API endpoint.

# Requirements:
# - Implement token-based authentication (e.g., JWT)
# - Validate all user inputs

```

```
# - Use proper HTTP methods (GET, POST only where needed)
# - Add rate limiting to prevent abuse
# - Return proper error responses
# - Follow API security best practices
```

Secure Flask API Endpoint

This Flask API application demonstrates several security best practices for API development:

- **Token-based Authentication (JWT):** Users authenticate to get a token, which is then used to access protected endpoints.
- **Input Validation:** User input is validated using `marshmallow` schemas.
- **Rate Limiting:** Requests are rate-limited using `Flask-Limiter` to prevent abuse.
- **Proper Error Responses:** Custom error handlers provide meaningful error messages.
- **HTTP Method Enforcement:** Endpoints use appropriate HTTP methods (e.g., `POST` for creating resources, `GET` for retrieving).

```
# Install necessary libraries for the Flask API
!pip install flask PyJWT marshmallow Flask-Limiter
```

```
Requirement already satisfied: flask in
/usr/local/lib/python3.12/dist-packages (3.1.3)
Requirement already satisfied: PyJWT in
/usr/local/lib/python3.12/dist-packages (2.12.1)
Requirement already satisfied: marshmallow in
/usr/local/lib/python3.12/dist-packages (4.3.0)
Requirement already satisfied: Flask-Limiter in
/usr/local/lib/python3.12/dist-packages (4.1.1)
Requirement already satisfied: blinker>=1.9.0 in
/usr/local/lib/python3.12/dist-packages (from flask) (1.9.0)
Requirement already satisfied: click>=8.1.3 in
/usr/local/lib/python3.12/dist-packages (from flask) (8.3.2)
Requirement already satisfied: itsdangerous>=2.2.0 in
/usr/local/lib/python3.12/dist-packages (from flask) (2.2.0)
Requirement already satisfied: jinja2>=3.1.2 in
/usr/local/lib/python3.12/dist-packages (from flask) (3.1.6)
Requirement already satisfied: markupsafe>=2.1.1 in
/usr/local/lib/python3.12/dist-packages (from flask) (3.0.3)
Requirement already satisfied: werkzeug>=3.1.0 in
/usr/local/lib/python3.12/dist-packages (from flask) (3.1.8)
Requirement already satisfied: limits>=3.13 in
/usr/local/lib/python3.12/dist-packages (from Flask-Limiter) (5.8.0)
Requirement already satisfied: ordered-set<5,>4 in
/usr/local/lib/python3.12/dist-packages (from Flask-Limiter) (4.1.0)
Requirement already satisfied: typing-extensions>=4.3 in
/usr/local/lib/python3.12/dist-packages (from Flask-Limiter) (4.15.0)
Requirement already satisfied: deprecated>=1.2 in
/usr/local/lib/python3.12/dist-packages (from limits>=3.13->Flask-
Limiter) (1.3.1)
```

Requirement already satisfied: packaging>=21 in
/usr/local/lib/python3.12/dist-packages (from limits>=3.13->Flask-Limiter) (26.0)

Requirement already satisfied: wrapt<3,>=1.10 in
/usr/local/lib/python3.12/dist-packages (from deprecated>=1.2->limits>=3.13->Flask-Limiter) (2.1.2)

Install necessary libraries for the Flask API

!pip install flask PyJWT marshmallow Flask-Limiter

Requirement already satisfied: flask in
/usr/local/lib/python3.12/dist-packages (3.1.3)

Requirement already satisfied: PyJWT in
/usr/local/lib/python3.12/dist-packages (2.12.1)

Collecting marshmallow

Downloading marshmallow-4.3.0-py3-none-any.whl.metadata (6.8 kB)

Collecting Flask-Limiter

Downloading flask_limiter-4.1.1-py3-none-any.whl.metadata (6.3 kB)

Requirement already satisfied: blinker>=1.9.0 in
/usr/local/lib/python3.12/dist-packages (from flask) (1.9.0)

Requirement already satisfied: click>=8.1.3 in
/usr/local/lib/python3.12/dist-packages (from flask) (8.3.2)

Requirement already satisfied: itsdangerous>=2.2.0 in
/usr/local/lib/python3.12/dist-packages (from flask) (2.2.0)

Requirement already satisfied: jinja2>=3.1.2 in
/usr/local/lib/python3.12/dist-packages (from flask) (3.1.6)

Requirement already satisfied: markupsafe>=2.1.1 in
/usr/local/lib/python3.12/dist-packages (from flask) (3.0.3)

Requirement already satisfied: werkzeug>=3.1.0 in
/usr/local/lib/python3.12/dist-packages (from flask) (3.1.8)

Collecting limits>=3.13 (from Flask-Limiter)

Downloading limits-5.8.0-py3-none-any.whl.metadata (10 kB)

Collecting ordered-set<5,>4 (from Flask-Limiter)

Downloading ordered_set-4.1.0-py3-none-any.whl.metadata (5.3 kB)

Requirement already satisfied: typing-extensions>=4.3 in
/usr/local/lib/python3.12/dist-packages (from Flask-Limiter) (4.15.0)

Collecting deprecated>=1.2 (from limits>=3.13->Flask-Limiter)

Downloading deprecated-1.3.1-py2.py3-none-any.whl.metadata (5.9 kB)

Requirement already satisfied: packaging>=21 in
/usr/local/lib/python3.12/dist-packages (from limits>=3.13->Flask-Limiter) (26.0)

Requirement already satisfied: wrapt<3,>=1.10 in
/usr/local/lib/python3.12/dist-packages (from deprecated>=1.2->limits>=3.13->Flask-Limiter) (2.1.2)

Downloading marshmallow-4.3.0-py3-none-any.whl (49 kB)
49.1/49.1 kB 1.8 MB/s eta

0:00:00

iter-4.1.1-py3-none-any.whl (30 kB)

Downloading limits-5.8.0-py3-none-any.whl (60 kB)
61.0/61.0 kB 3.8 MB/s eta

```

0:00:00
marshmallow, deprecated, limits, Flask-Limiter
Successfully installed Flask-Limiter-4.1.1 deprecated-1.3.1 limits-
5.8.0 marshmallow-4.3.0 ordered-set-4.1.0

import os
from datetime import datetime, timedelta
import jwt
from flask import Flask, request, jsonify
from marshmallow import Schema, fields, ValidationError
from flask_limiter import Limiter
from flask_limiter.util import get_remote_address

# --- Configuration ---
app = Flask(__name__)

# Secret key for JWT (should be a strong, random key in production)
app.config['SECRET_KEY'] = 'your_super_secret_api_key_for_jwt'
app.config['JWT_EXPIRATION_DAYS'] = 1 # Token expires in 1 day

# --- Rate Limiting Configuration ---
# Apply a default rate limit of 200 requests per day per IP address
limiter = Limiter(
    get_remote_address,
    app=app,
    default_limits=["200 per day", "50 per hour"]
)

# Simple user store for demonstration (in a real app, use a database)
USERS = {
    'user1': 'securepass123',
    'admin': 'adminpass456'
}

print('Flask app and initial configurations loaded.')

Flask app and initial configurations loaded.

/usr/local/lib/python3.12/dist-packages/flask_limiter/
_extension.py:364: UserWarning: Using the in-memory storage for
tracking rate limits as no storage was explicitly specified. This is
not recommended for production use. See: https://flask-
limiter.readthedocs.io#configuring-a-storage-backend for documentation
about configuring the storage backend.
  warnings.warn(

```

JWT (JSON Web Token) Functions

These functions handle the creation and verification of JWTs. Tokens are signed using the SECRET_KEY and include an expiration time.

```

def generate_token(username):
    """Generates a JWT for a given username."""
    expiration_time = datetime.utcnow() +
timedelta(days=app.config['JWT_EXPIRATION_DAYS'])
    payload = {
        'username': username,
        'exp': expiration_time
    }
    token = jwt.encode(payload, app.config['SECRET_KEY'],
algorithm='HS256')
    return token

def verify_token(token):
    """Verifies a JWT and returns the payload if valid, otherwise
None."""
    try:
        payload = jwt.decode(token, app.config['SECRET_KEY'],
algorithms=['HS256'])
        return payload
    except jwt.ExpiredSignatureError:
        return {'error': 'Token has expired'}
    except jwt.InvalidTokenError:
        return {'error': 'Invalid token'}

print('JWT functions defined.')
JWT functions defined.

```

Input Validation with Marshmallow

`marshmallow` is used to define schemas for expected request data, ensuring that incoming data conforms to the required format and types. This prevents common injection and data manipulation vulnerabilities.

```

# --- Input Validation Schemas ---
class LoginSchema(Schema):
    username = fields.String(required=True)
    password = fields.String(required=True)

class DataSchema(Schema):
    id = fields.Integer(required=True)
    name = fields.String(required=True)
    value = fields.String()

print('Marshmallow schemas defined.')
Marshmallow schemas defined.

```

API Endpoints and Error Handling

This section defines the API endpoints, including a login endpoint to get a JWT, and a protected endpoint that requires a valid token. Custom error handlers are implemented for validation and authentication failures.

```
# --- Error Handlers ---
@app.errorhandler(400)
def bad_request(error):
    return jsonify({'error': 'Bad Request', 'message': str(error)}),
    400

@app.errorhandler(401)
def unauthorized(error):
    return jsonify({'error': 'Unauthorized', 'message':
    'Authentication required or invalid token'}), 401

@app.errorhandler(403)
def forbidden(error):
    return jsonify({'error': 'Forbidden', 'message': 'You do not have
    permission to access this resource'}), 403

@app.errorhandler(404)
def not_found(error):
    return jsonify({'error': 'Not Found', 'message': 'The requested
    URL was not found on the server'}), 404

@app.errorhandler(429)
def ratelimit_handler(e):
    return jsonify({'error': 'Too Many Requests', 'message': 'Rate
    limit exceeded. Try again later.'}), 429

# --- Decorator for Token Required ---
def token_required(f):
    def decorated(*args, **kwargs):
        token = None
        if 'x-access-token' in request.headers:
            token = request.headers['x-access-token']
        if not token:
            return jsonify({'message': 'Token is missing!'}), 401

        current_user_payload = verify_token(token)
        if not current_user_payload or 'error' in
        current_user_payload:
            return jsonify({'message':
            current_user_payload.get('error', 'Token is invalid!')}), 401

        return f(current_user_payload['username'], *args, **kwargs)
    return decorated
```

```

# --- API Endpoints ---
@app.route('/login', methods=['POST'])
def login():
    schema = LoginSchema()
    try:
        data = schema.load(request.json)
    except ValidationError as err:
        return jsonify(err.messages), 400

    username = data.get('username')
    password = data.get('password')

    if username in USERS and USERS[username] == password:
        token = generate_token(username)
        return jsonify({'token': token}), 200
    return jsonify({'message': 'Invalid credentials'}), 401

@app.route('/protected', methods=['GET'])
@token_required
@limiter.limit("10 per minute") # Specific rate limit for this endpoint
def protected_resource(current_user):
    return jsonify({'message': f'Hello, {current_user}! You accessed a protected resource.'}), 200

@app.route('/data', methods=['POST'])
@token_required
def submit_data(current_user):
    schema = DataSchema()
    try:
        data = schema.load(request.json)
    except ValidationError as err:
        return jsonify(err.messages), 400

    # In a real application, you would save this data to a database
    print(f"User {current_user} submitted data: {data}")
    return jsonify({'message': 'Data received successfully', 'data': data}), 201

@app.route('/', methods=['GET'])
def home():
    return jsonify({'message': 'Welcome to the Secure Flask API. Try /login or /protected.'}), 200

print('API endpoints and error handlers defined.')

```

How to run the Flask API in Colab:

1. **Install necessary libraries:** `python !pip install flask PyJWT marshmallow Flask-Limiter` You might also need `flask-ngrok` to expose the API publicly. `python !pip install flask-ngrok`
2. **Modify the Flask app execution** to use `flask-ngrok` (if you want to access it from your browser/external tools). You'll need to add `from flask_ngrok import run_with_ngrok` and call `run_with_ngrok(app)` before `app.run()`.

```
# Example of modified Flask app execution
from flask_ngrok import run_with_ngrok

if __name__ == '__main__':
    run_with_ngrok(app) # Starts ngrok when app is run
    app.run()
```

After running this modified cell, `flask-ngrok` will provide a public URL that you can use to interact with your API (e.g., using `curl`, Postman, or a web browser).

Alternatively, if you prefer not to use ngrok and run it locally (e.g., if you're running Colab locally):

```
if __name__ == '__main__':
    app.run(debug=True, port=5000)
```

3. Interact with the API:

- **Login (POST):** Send a POST request to `/login` with `{"username": "user1", "password": "securepass123"}` in the request body to get a JWT.
- **Protected Resource (GET):** Send a GET request to `/protected`, including the JWT in the `x-access-token` header (e.g., `x-access-token: <your_jwt_here>`).
- **Submit Data (POST):** Send a POST request to `/data` with a valid JWT and data like `{"id": 1, "name": "Test Item", "value": "some_value"}`.